

Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут інформаційно-комунікаційних технологій та електронної інженерії
Кафедра «Інформаційно-комунікаційних технологій»



Звіт з лабораторної роботи №6
з дисципліни «Об'єктно-орієнтоване програмування, частина 2»

Підготував:
ст. групи IX-21
Петриченко Сергій

Прийняла:
док. філософії, доцент
Гордійчук-Бублівська О.В.

Львів 2026 р.

Тема роботи: Реалізація клієнт-серверної взаємодії з використанням REST API.

Мета роботи: Розроблення клієнт-серверної архітектури з використанням REST API та практичне застосування класів для реалізації взаємодії між компонентами системи.

Теоретичні відомості

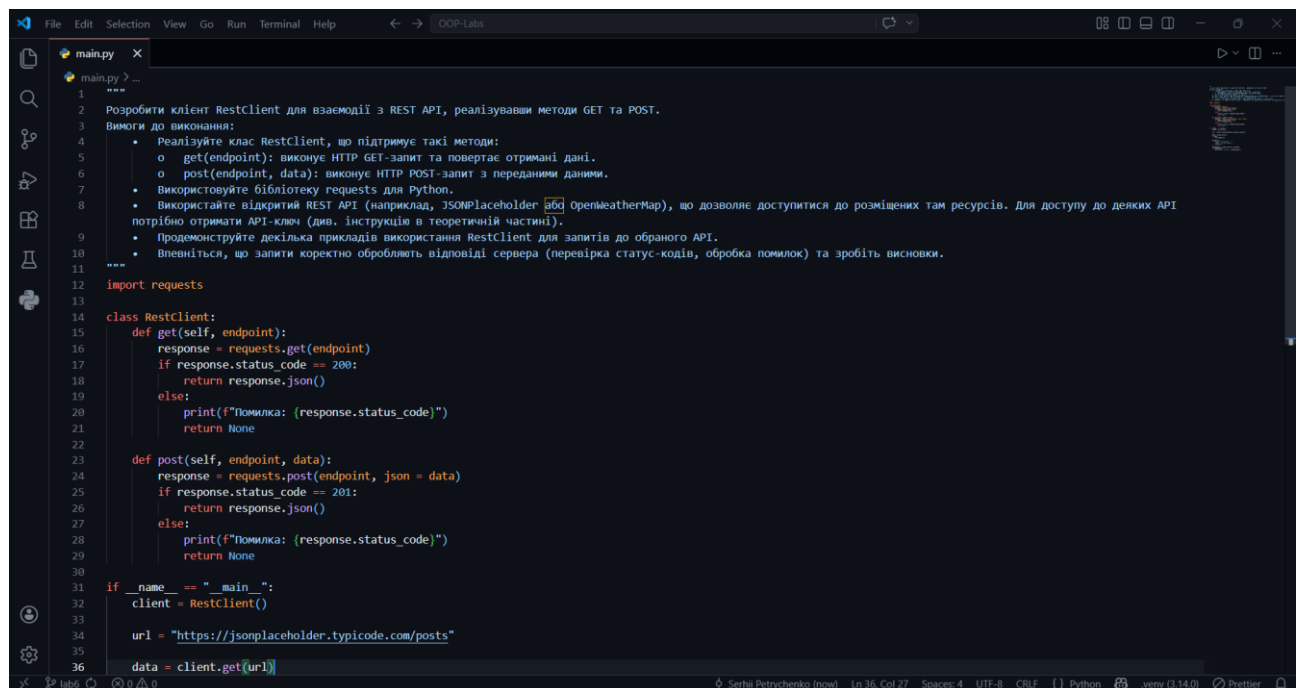
Клієнт-серверна архітектура є основою роботи сучасного інтернету, де клієнт надсилає запити до сервера, а той обробляє їх і надсилає відповідь.

REST API (Representational State Transfer Application Programming Interface) – це архітектурний стиль для взаємодії між компонентами системи через протокол HTTP. Взаємодія базується на таких принципах:

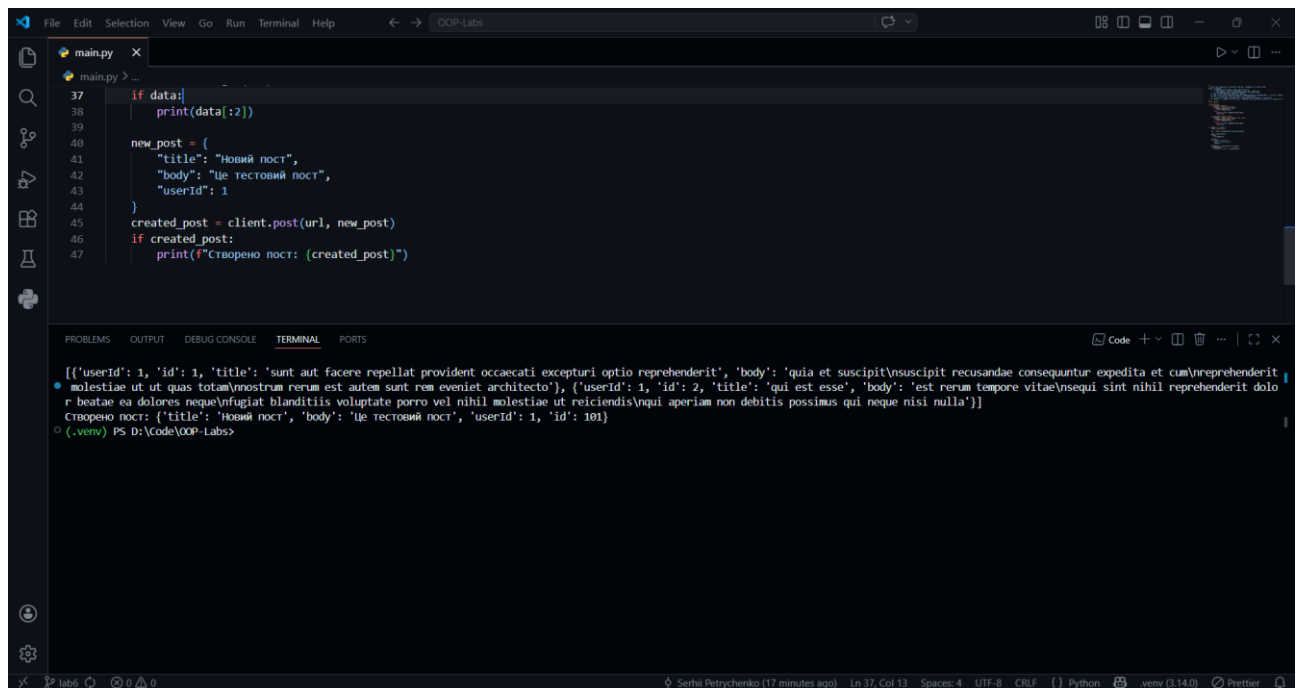
- Методи HTTP: використовується GET для отримання даних, POST для створення нових записів, PUT/PATCH для оновлення та DELETE для видалення.
- Endpoint (Кінцева точка): унікальна URL-адреса, до якої звертається клієнт.
- Формати даних: найчастіше для обміну інформацією використовується формат JSON.
- Авторизація: багато сервісів потребують використання API-ключа, який передається в URL або заголовках запиту.

Тестування таких систем зручно проводити за допомогою фіктивних сервісів, як-от JSONPlaceholder, що імітують роботу реального сервера.

Результати виконаної роботи



```
1  """
2  Розробити клієнт RestClient для взаємодії з REST API, реалізуючи методи GET та POST.
3  Вимоги до виконання:
4  * Реалізуйте клас RestClient, що підтримує такі методи:
5    o get(endpoint): виконує HTTP GET-запит та повертає отримані дані.
6    o post(endpoint, data): виконує HTTP POST-запит з переданими даними.
7  * Використовуйте бібліотеку requests для Python.
8  * Використайте відкритий REST API (наприклад, JSONPlaceholder або OpenWeatherMap), що дозволяє доступитися до розміщених там ресурсів. Для доступу до деяких API
9    потрібно отримати API-ключ (див. інструкцію в теоретичній частині).
10 * Продемонструйте декілька прикладів використання RestClient для запитів до обраного API.
11 * Впевніться, що запити коректно обробляють відповіді сервера (перевірка статус-кодів, обробка помилок) та зробіть висновки.
12 """
13
14 import requests
15
16 class RestClient:
17     def get(self, endpoint):
18         response = requests.get(endpoint)
19         if response.status_code == 200:
20             return response.json()
21         else:
22             print(f"Помилка: {response.status_code}")
23             return None
24
25     def post(self, endpoint, data):
26         response = requests.post(endpoint, json = data)
27         if response.status_code == 201:
28             return response.json()
29         else:
30             print(f"Помилка: {response.status_code}")
31             return None
32
33 if __name__ == "__main__":
34     client = RestClient()
35
36     url = "https://jsonplaceholder.typicode.com/posts"
37     data = client.get(url)
```



The screenshot shows a VS Code editor window with a file named `main.py`. The code defines a `RestClient` class with `get` and `post` methods. The `post` method is used to create a new post. The terminal output shows the response from the API, which is a JSON object containing the created post details.

```
37 if data:
38     print(data[2])
39
40 new_post = {
41     "title": "Новий пост",
42     "body": "Це тестовий пост",
43     "userId": 1
44 }
45 created_post = client.post(url, new_post)
46 if created_post:
47     print(f"Створено пост: {created_post}")
```

```
[[{"userId": 1, "id": 1, "title": 'sunt aut facere repellat provident occaecati excepturi optio reprehenderit', 'body': 'quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto'}, {"userId": 1, "id": 2, "title": 'qui est esse', 'body': 'est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis\nqui aperiam non debitis possimus qui neque nisi nulla'}]]
Створено пост: {'title': 'Новий пост', 'body': 'Це тестовий пост', 'userId': 1, 'id': 101}
```

Висновки

1. У ході роботи було розроблено клас `RestClient` на мові Python із використанням бібліотеки `requests`. Реалізовано методи `get` для отримання даних та `post` для надсилання інформації на сервер. Роботу клієнта перевірено на прикладі відкритого API `JSONPlaceholder`.
2. Такий клієнт можна інтегрувати в будь-який програмний проект, де потрібна взаємодія із зовнішніми сервісами. Наприклад, для отримання актуальних даних про погоду через `OpenWeatherMap` або для автоматизації збору публічної інформації з веб-ресурсів.
3. Практичне застосування класів дозволило глибше зрозуміти механізми обробки HTTP-запитів. Отримано навички роботи зі статус-кодами (як-от 200 OK або 201 Created), обробки JSON-структур та налаштування заголовків авторизації.
4. Переваги: REST API забезпечує стандартизований та зрозумілий спосіб обміну даними між різними платформами. Використання бібліотеки `requests` значно спрощує написання коду порівняно з низькорівневими методами.
Недоліки: робота системи критично залежить від стабільності інтернет-з'єднання та доступності віддаленого сервера. Також використання багатьох API обмежене необхідністю отримання та безпечного зберігання API-ключів.